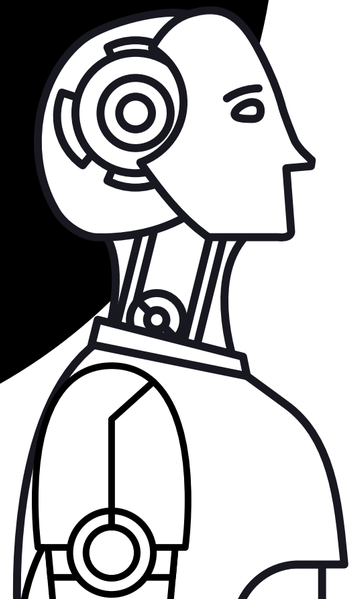
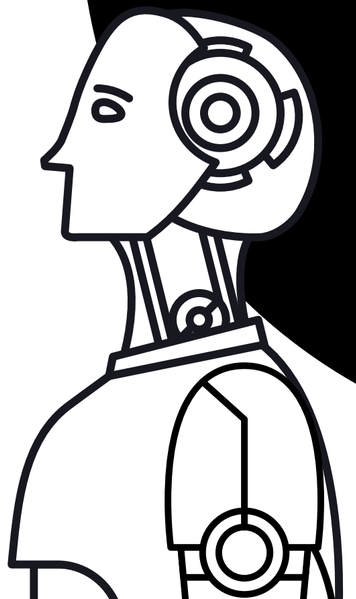




ALGORITMOS GULOSOS (GREEDY ALGORITHMS): DA OTIMIZAÇÃO LOCAL AO ÓTIMO GLOBAL

A P R E S E N T A Ç Ã O

Integrantes:
Arthur Ribeiro Gonçalves,Davi Enzo Vieira,Felipe Côrtes do Nascimento,Felipe Silva,Pedro Francisco Sousa Silva,Pedro Henrique Silva Costa

A ESSÊNCIA DO ALGORITMO GULOSO

- Busca resolver problemas de otimização construindo uma solução passo a passo

- O algoritmo sempre toma a opção que parece melhor em cada etapa do funcionamento (ótimo local)

- A meta principal dele é que a soma dos otimos locais resulte na melhor solução do problema (otimo global)



O INSTINTO GULOSO NA PRÁTICA

PROBLEMAS MOTIVADORES:

Exemplo Intuitivo: Como dar R$ 0,85 de troco usando o menor numero de moedas (50¢, 25¢, 10¢, 5¢)?

Ação Gulosa: Pegar sempre a maior moeda que cabe agora (50¢→+25¢→+10¢=3 moedas).

ONDE ISSO RESOLVE PROBLEMAS GIGANTES NA COMPUTAÇÃO?



01 "Como comprimir um arquivo de texto ou imagem no menor número de bits possível?" (ZIP / JPEG)

02 "Como conectar dezenas de cidades com fibra óptica gastando o mínimo de cabo?" (Redes)

03 Como alocar o máximo de processos em uma única CPU sem conflito de horário?" (Sistemas Operacionais)

POR QUE SER GULOSO? SUPERANDO LIMITAÇÕES CLÁSSICAS

01

FORÇA BRUTA

Avalia todas as soluções possíveis

Explosão Combinatória ($O(2^n)$ ou $O(n!)$). Inviável!



02

PROGRAMAÇÃO DINAMICA

Resolve todos os subproblemas antes de escolher e guarda tabelas na memória.

Garante o ótimo, mas cobra um alto preço de memória e processamento.

03

ALGORITMO GULOSO

Não calcula ramificações futuras! Faz a escolha local imediata e resolve apenas um caminho.

Resultado: Algoritmos extremamente rápidos (geralmente $\Theta(n \log n)$) e com gasto mínimo de memória.

PILARES TEÓRICOS E ALGORITMOS CLÁSSICOS

QUANDO AGIR DE FORMA "MÍOPE" GARANTE 100% DE CERTEZA DO ÓTIMO GLOBAL?

- SUBESTRUTURA ÓTIMA, PROPRIEDADE DA ESCOLHA GULOSA E TEORIA DOS MATROIDES.

- Os 4 Algoritmos Clássicos que vamos construir e analisar hoje:
  - Seleção de Atividades (Escalonamento de intervalos)
  - Mochila Fracionária (Otimização contínua vs. Mochila 0-1)
  - Codificação de Huffman (Compressão ótima de dados)
  - Kruskal e Prim (Árvores Geradoras Mínimas em Grafos)



FUNDAMENTAÇÃO TEÓRICA: O PARADIGMA GULOSO



O DILEMA DE OTIMIZAÇÃO:

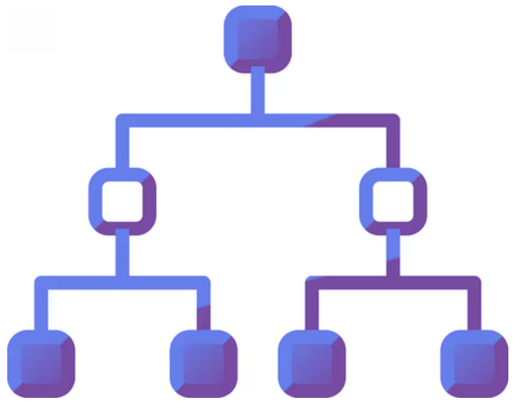
- **Força Bruta:** Impraticável (explosão combinatória).
- **Programação Dinâmica:** Exata, mas exige calcular todos os subproblemas e alta memória.
- **Algoritmo Guloso:** Decisões locais imediatas, irrevogáveis e sem backtracking.
- **O Questionamento Central:** Quando uma sequência de decisões míopes garante o ótimo global?
- **Condições Suficientes:** Subestrutura Ótima + Propriedade da Escolha Gulosa.

SUBESTRUTURA ÓTIMA (OPTIMAL SUBSTRUCTURE)

DEFINIÇÃO FORMAL: UMA SOLUÇÃO ÓTIMA GLOBAL CONTÉM EM SEU INTERIOR SOLUÇÕES ÓTIMAS DE SUBPROBLEMAS RESIDUAIS.

RELAÇÃO RESIDUAL:

$$f(Y \cup \{c\}) = \varphi(c) + f_c(Y)$$

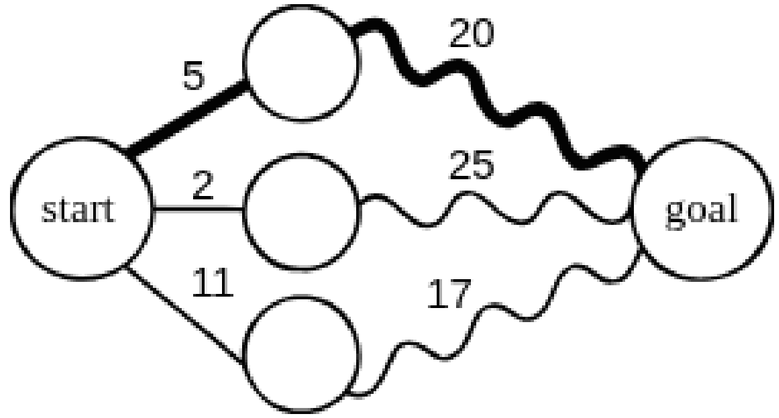


A PROVA: RECORTAR E COLAR (CUT-AND-PASTE):

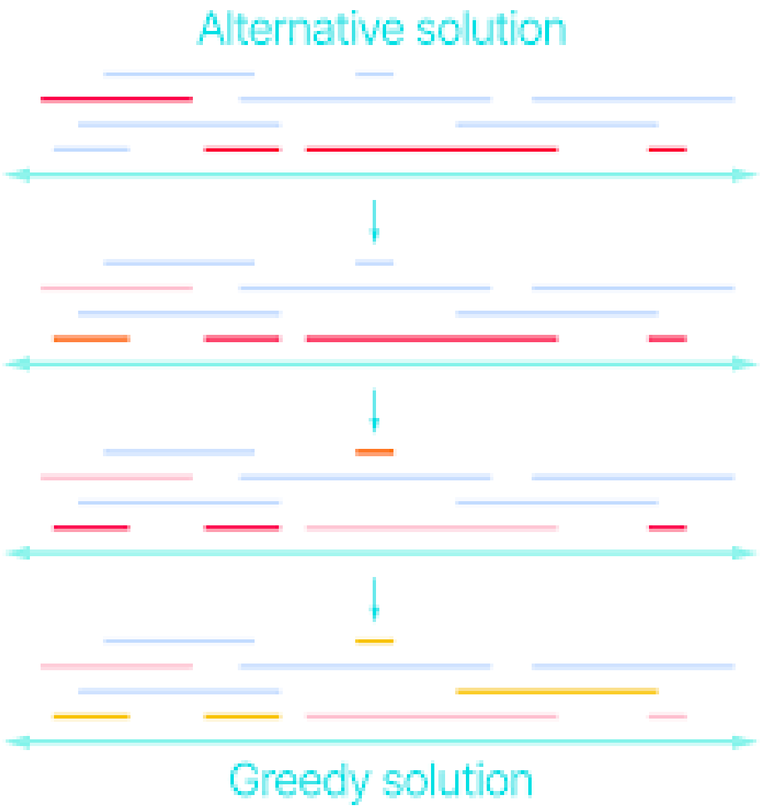
- Supõe-se por absurdo que o resíduo $\mathbf{X \setminus \{c\}}$ não é ótimo.
- Se existisse **Y** melhor no resíduo, a união $\mathbf{Y \cup \{c\}}$ superaria a solução global ótima inicial, gerando contradição.

CONTRASTE ESTRUTURAL:

- **Prog. Dinâmica:** $\max^c \{\varphi(c) + opt(\Pi^c)\} \rightarrow$ Múltiplas ramificações (bottom-up).
- **Guloso:** $c^g = \arg\max \kappa(c) \rightarrow$ Ramo único descendente (top-down).



PROPRIEDADE DA ESCOLHA GULOSA (GREEDY-CHOICE PROPERTY)



CONDIÇÃO EXISTENCIAL:

$$\exists X^* \in OPT(\Pi) \text{ tal que } c^g \in X^*$$

- A decisão local nunca invalida a conquista do ótimo global.

TÉCNICA ANALÍTICA: ARGUMENTO DE TROCA (EXCHANGE ARGUMENT):

$$X' = (X^* \setminus \{c'\}) \cup \{c^g\}$$

Preserva a viabilidade: $X' \in \mathscr{F}$.

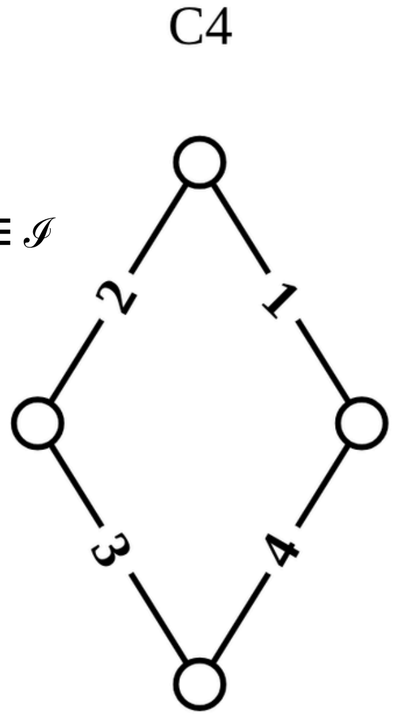
Preserva o valor: $f(X') \geqslant f(X^*)$.

- **Exemplo Clássico:** Seleção de Atividades (a atividade que termina primeiro sempre integra uma solução máxima).
- **Onde falha:** Mochila 0-1 (a troca de itens rígidos pode deixar peso ocioso ou estourar a capacidade).

TEORIA DOS MATROIDES (WHITNEY, RADO E EDMONDS)

DEFINIÇÃO AXIOMÁTICA: M = (S, $\mathscr{I}$)

- **(M1)** Conjunto finito de elementos S.
- **(M2)** Propriedade Hereditária: $B \in \mathscr{I} \land A \subseteq B \Rightarrow A \in \mathscr{I}$.
- **(M3)** Propriedade da Troca (Aumento): $\mathbf{A, B \in \mathscr{I} \land |A| < |B| \Rightarrow \exists x \in B \setminus A : A \cup \{x\} \in \mathscr{I}}$



C(3, 4)
Independent Sets
I0: {}
I1: {1}
I2: {2}
I3: {3}
I4: {4}
I5: {1, 2}
I6: {1, 3}
I7: {1, 4}
I8: {2, 3}
I9: {2, 4}
I10: {3, 4}
I11: {1, 2, 3}
I12: {1, 2, 4}
I13: {1, 3, 4}
I14: {2, 3, 4}

CONCEITOS FUNDAMENTAIS:

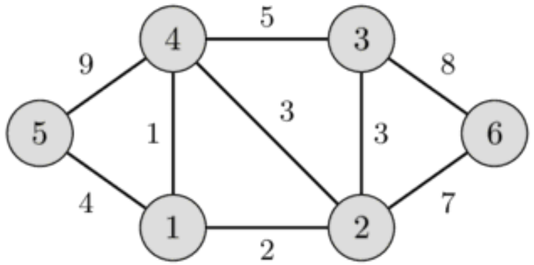
- **Bases:** Independentes maximais; todas têm estritamente a mesma cardinalidade (posto r(S)).
- **Circuitos:** Dependentes minimais (adicionar um elemento a uma base cria exatamente um circuito fundamental).

TEOREMA DE RADO-EDMONDS E MATROIDES GRÁFICOS

TEOREMA CENTRAL (RADO-EDMONDS):

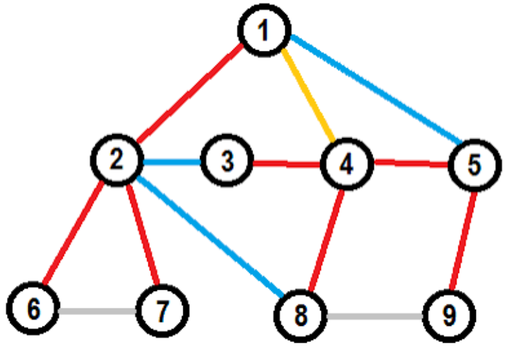
- O algoritmo guloso genérico encontra uma base de peso máximo para toda função de peso estritamente positiva se, e somente se, o sistema $(S, \mathscr{I})$ for um Matroide.

APLICAÇÃO EM GRAFOS (MATROIDE GRÁFICO M^g):



- Elementos: S = E (arestas).
- Independentes: $\mathscr{I}$ = {A ⊆ E : (V, A) é acíclico}.
- Bases: Árvores Geradoras (|V| − 1 arestas).

PONTE COM O ALGORITMO DE KRUSKAL (COROLÁRIO 1):



- O matroide maximiza; Kruskal minimiza
- Transformação afim: $w(e) = w_0 - y(e)$, com $w_0 > \max y(e)$.
- A ordem decrescente de w equivale exatamente à ordem crescente de custos de Kruskal.

O PARADIGMA ALGORÍTMICO

Exemplo de funcionamento atraves do exemplo do Cormen:

Algoritmo 1: GREEDY(M, w) – Esqueleto Universal [1]

Entrada: Matroide $M = (S, \mathcal{I})$ e função de peso $w : S \to \mathbb{R}^+$
Saída : Subconjunto viável ótimo $A \in M.\mathcal{I}$ de peso máximo

```
1 A ← ∅
  // 1. Pré-processamento (Ordenação)
2 Ordene M.S em ordem monotonicamente decrescente pelo peso w
  // 2. Laço de Seleção Local
3 para cada x ∈ M.S, tomado em ordem decrescente de w(x) faça
  |   // 3. Teste de Viabilidade
4 |   se A ∪ {x} ∈ M.I então
5 |   |   A ← A ∪ {x}
6 retorna A
```



USO DE MATROIDE

Padrão do algoritmo para garantir melhor global, além de somente local.

01

PRÉ-PROCESSAMENTO

Iniciar o conjunto de soluções vazio e ordenar, decrescente, candidatos do algoritmo.

02

LAÇO DE SELEÇÃO

Percorre a lista de candidatos para fazer a validação um por vez.

03

VALIDAÇÃO

Através da regra de condição estabelecida, verifica se aquele candidato pode ser incluido na solução

EXEMPLO FUNCIONAL

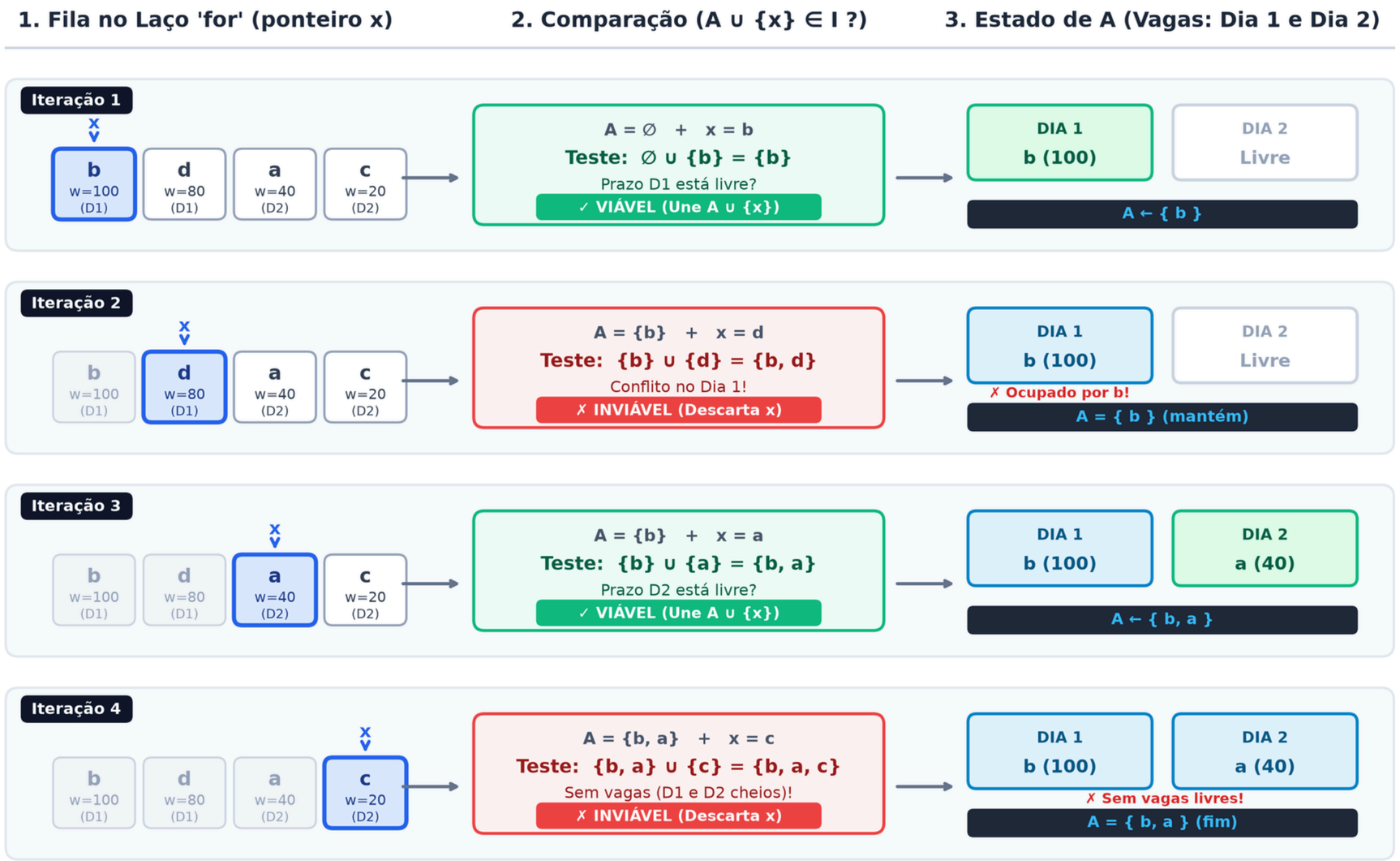
CONDIÇÃO E ENTRADA INICIAL:
MATROIDE

QUAL O MELHOR CONJUNTO DE TAREFAS É POSSÍVEL FAZER?



EXEMPLO FUNCIONAL

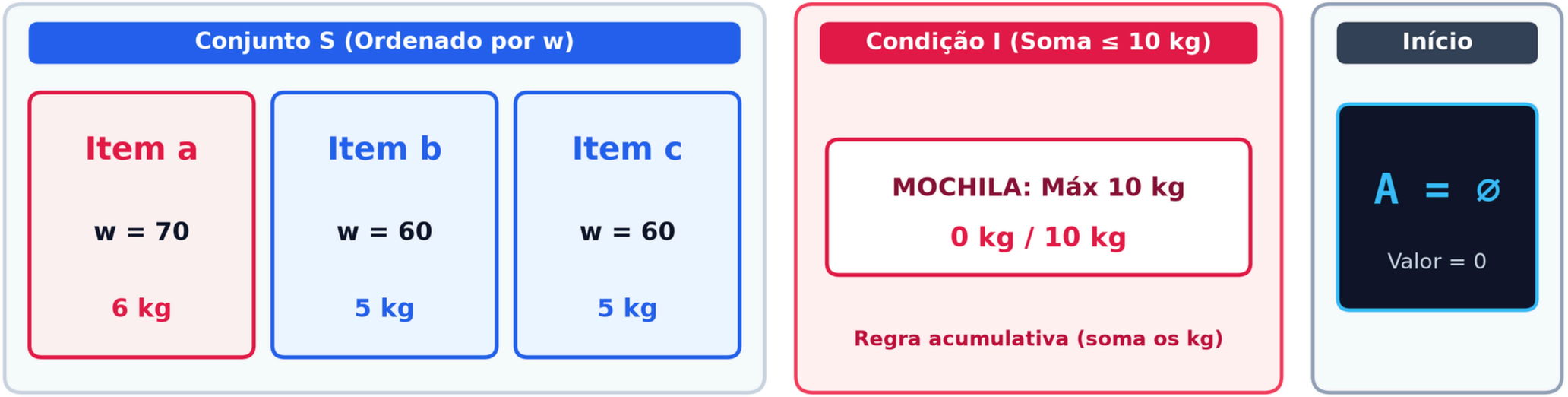
INTERAÇÕES



EXEMPLO FUNCIONAL

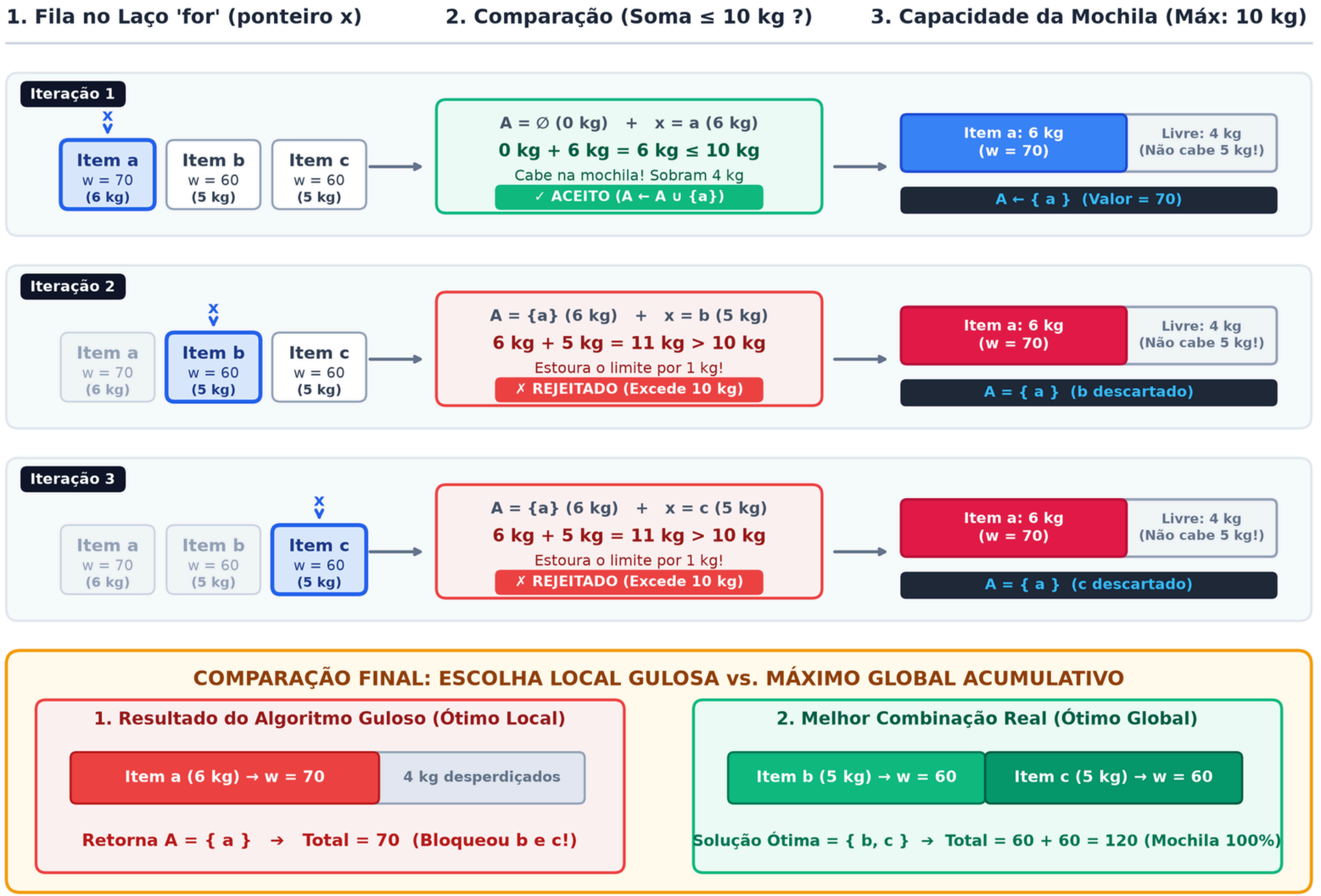
CONDIÇÃO E ENTRADA INICIAL:
NÃO MATROIDE

MOCHILA DO CASTOR VIAJANTE



EXEMPLO FUNCIONAL

INTERAÇÕES



VANTAGEM DO INTERATIVO

Como o Algoritmo guloso não observa alternativas de resultado além da sequência, ele apresenta algumas vantagens ao usar o método Interativo

• **Desvantagem da Recursão:** Gasto desnecessário na gestão das pilhas de função (Call Stack), consumindo memória O(n) que pode sair do controle (Stack Overflow).

• **Desvantagem da Programação Dinâmica:** Algoritmos que testam todas as combinações para achar o ótimo global precisam armazenar resultados em matrizes, o que pode estourar a memória.

• **A Vantagem do Iterativo Guloso:** Como a escolha reduz o problema a um único subproblema por vez (Recursão de Cauda), ela vira um simples laço for/while: guarda apenas o conjunto atual A, verifica cada candidato uma única vez e economiza pilhas e matrizes (Memória auxiliar constante Θ(1)), embora o armazenamento da solução final A possa ocupar O(n) ou O(k) espaço.



O “MOTOR” GULOSO

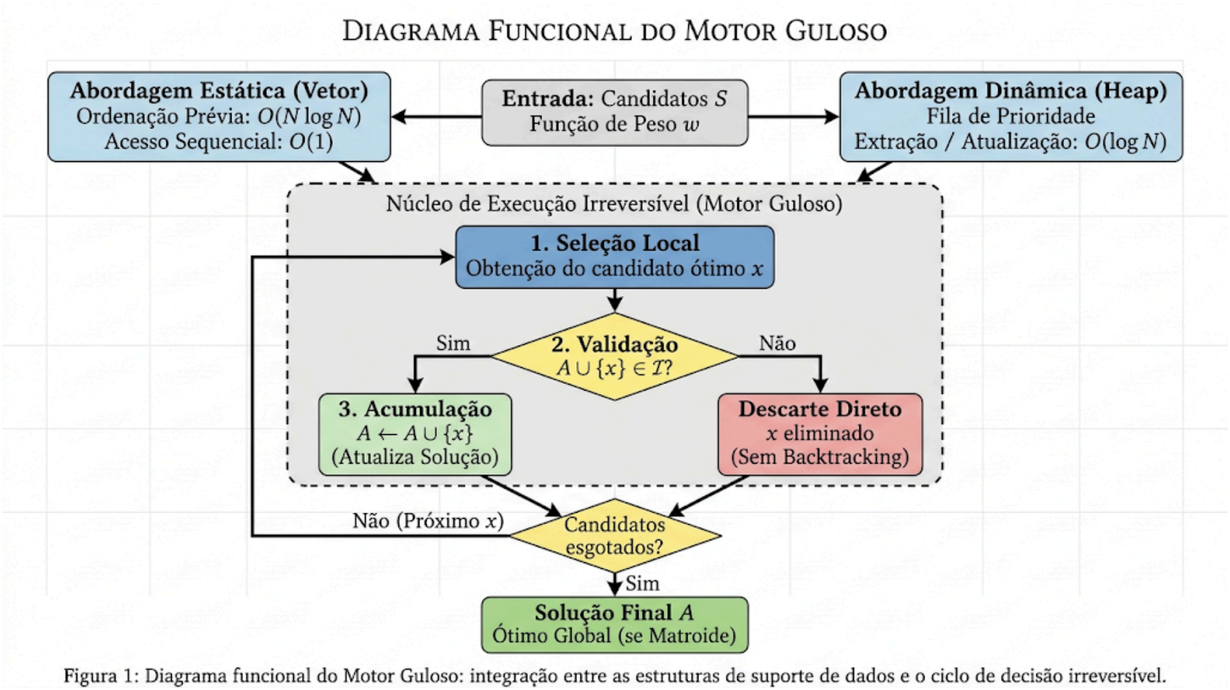


Figura 1: Diagrama funcional do Motor Guloso: integração entre as estruturas de suporte de dados e o ciclo de decisão irreversível.

A eficiência de um algoritmo guloso não está so na sua heurística de seleção, mas principalmente na estrutura de dados responsável por gerenciar o espaço de candidatos e fornecer o elemento ótimo a cada iteração. Para manter a escolha gulosa eficiente na memória e no tempo de execução, a engenharia da solução adota duas abordagens arquiteturais distintas, a estática e a dinâmica. A expressão "motor guloso" é apenas uma abstração didática utilizada para descrever a mecânica do laço de seleção.

01

ESTÁTICA

- Indicada para cenários em que os pesos e custos dos candidatos são fixos e totalmente conhecidos desde o início.
- Realiza uma única ordenação prévia do vetor de candidatos no início da execução.
- Após o custo inicial de O(N log N) na ordenação, a seleção da melhor opção a cada passo opera em tempo O(1) avançando um ponteiro, com alta localidade de cache e baixo consumo de memória.
- Algoritmo de Kruskal e Seleção de Atividades.

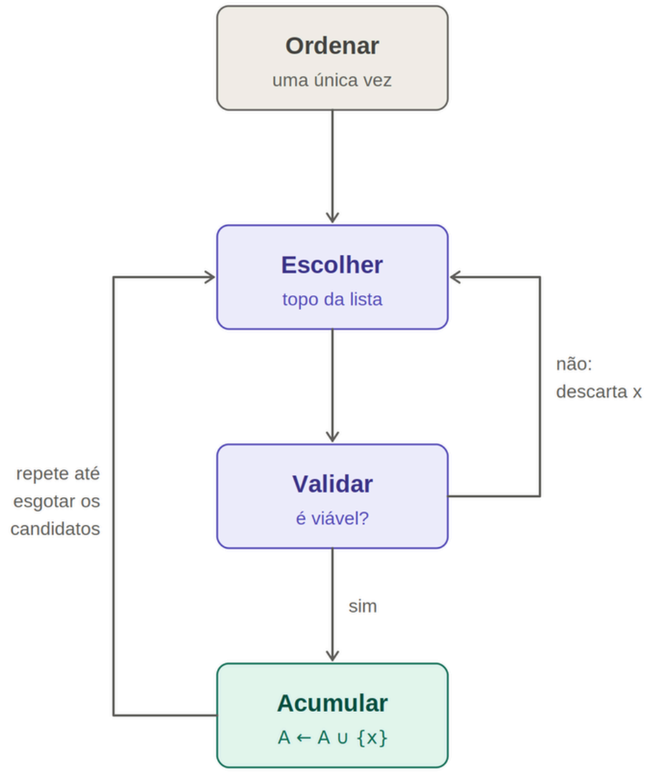
02

DINÂMICA

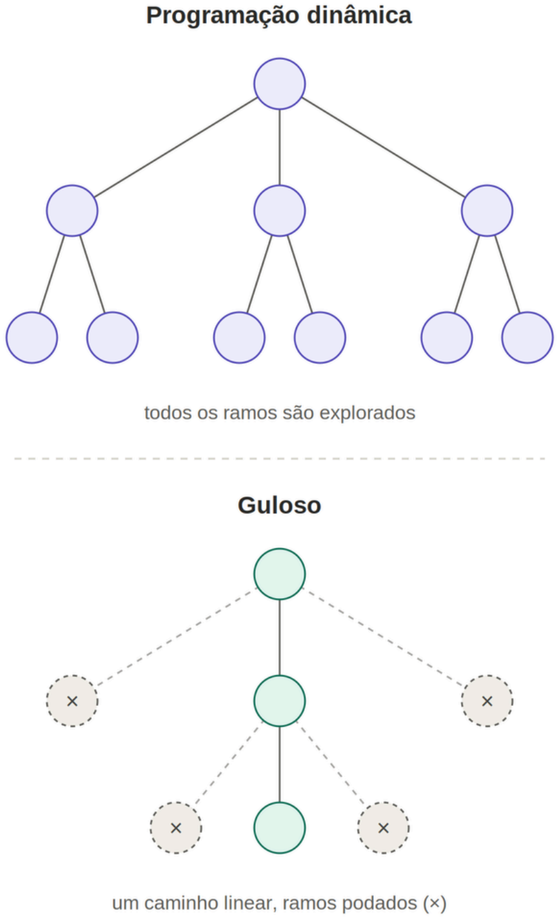
- Obrigatória quando a escolha de um elemento altera o custo dos demais ou insere novos candidatos na disputa.
- Mantém os elementos organizados em uma Fila de Prioridade (Min-Heap ou Max-Heap).
- Evita reordenações completas e custosas, permitindo consultar o ótimo local na raiz em O(1) e atualizar pesos ou extrair itens em tempo logarítmico O(log N).
- Algoritmo de Prim, Dijkstra e Códigos de Huffman.

DIAGRAMAS GERAIS

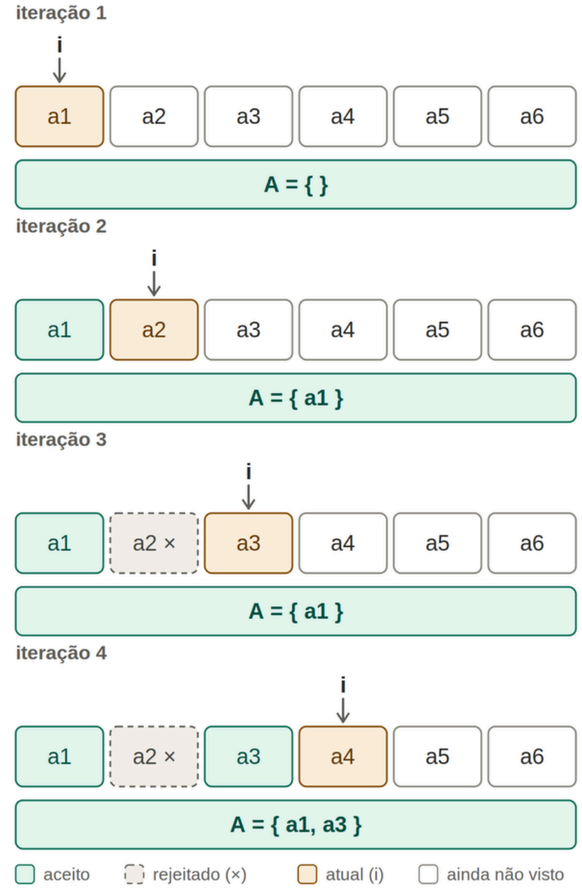
1. Fluxograma do ciclo de decisão gulosa



2. Árvore de recursão comparativa



3. Estado da memória: apenas A e o ponteiro i



ALGORITMO 1: SELEÇÃO DE ATIVIDADES

```
ALGORITMO SelecaoAtividades(atividades)
ENTRADA: Lista de atividades com horários [início, término]
SAÍDA:   Conjunto máximo de atividades compatíveis
1.  Ordene as atividades em ordem crescente de término
2.  escolhidas ← conjunto vazio
3.  horario_livre ← 0
4.  PARA CADA atividade DA lista_ordenada FAÇA:
5.      SE atividade.início ≥ horario_livre ENTÃO:
6.          Adicione atividade ao conjunto escolhidas
7.          horario_livre ← atividade.término
8.  RETORNE escolhidas
```

- **Contexto: Escalonamento de recursos compartilhados (como uma única sala de reunião ou um único núcleo de processador) disputados por diversas tarefas que possuem horários fixos de início e fim.**

- **Objetivo: Selecionar o maior número possível de atividades compatíveis, ou seja, que não tenham sobreposição de horário entre si.**

ALGORITMO 1: SELEÇÃO DE ATIVIDADES

```
ALGORITMO SelecaoAtividades(atividades)
ENTRADA: Lista de atividades com horários [início, término]
SAÍDA:   Conjunto máximo de atividades compatíveis
1.  Ordene as atividades em ordem crescente de término
2.  escolhidas ← conjunto vazio
3.  horario_livre ← 0
4.  PARA CADA atividade DA lista_ordenada FAÇA:
5.      SE atividade.início ≥ horario_livre ENTÃO:
6.          Adicione atividade ao conjunto escolhidas
7.          horario_livre ← atividade.término
8.  RETORNE escolhidas
```

Escolha Gulosa:

- **Critério: Escolher sempre a próxima atividade compatível que possui o menor horário de término.**

- **Por que funciona? Ao terminar o mais rápido possível, ela libera o recurso mais cedo, deixando o maior intervalo de tempo restante livre para acomodar as próximas atividades. Não é um matroide geral, é um sistema de independência**

ALGORITMO 2: MOCHILA FRACIONÁRIA

```
ALGORITMO MochilaFracionaria(itens, capacidade)
ENTRADA: Itens com valor e peso; capacidade máxima da mochila
SAÍDA:   Valor máximo acumulado
1.  PARA CADA item DA lista_itens FAÇA:
2.      item.densidade ← item.valor / item.peso
3.  Ordene os itens em ordem decrescente de densidade
4.  valor_total ← 0
5.  PARA CADA item DA lista_ordenada ENQUANTO capacidade > 0 FAÇA:
6.      peso_coletado ← MÍNIMO(capacidade, item.peso)
7.      fração ← peso_coletado / item.peso
8.      valor_total ← valor_total + (fração * item.valor)
9.      capacidade ← capacidade - peso_coletado
10. RETORNE valor_total
```

- **Contexto: Alocação de carga sob uma restrição contínua de capacidade (como um caminhão ou mochila com limite de peso em kg), onde os itens podem ser cortados/fracionados (grãos, líquidos, metais preciosos).**
- **Objetivo: Preencher a mochila respeitando o limite máximo de peso W, maximizando o valor monetário total transportado.**

ALGORITMO 2: MOCHILA FRACIONÁRIA

```
ALGORITMO MochilaFracionaria(itens, capacidade)
ENTRADA: Itens com valor e peso; capacidade máxima da mochila
SAÍDA:   Valor máximo acumulado
1.  PARA CADA item DA lista_itens FAÇA:
2.      item.densidade ← item.valor / item.peso
3.  Ordene os itens em ordem decrescente de densidade
4.  valor_total ← 0
5.  PARA CADA item DA lista_ordenada ENQUANTO capacidade > 0 FAÇA:
6.      peso_coletado ← MÍNIMO(capacidade, item.peso)
7.      fração ← peso_coletado / item.peso
8.      valor_total ← valor_total + (fração * item.valor)
9.      capacidade ← capacidade - peso_coletado
10. RETORNE valor_total
```

Escolha Gulosa

- **Critério: Selecionar os itens em ordem decrescente de densidade de valor (di=valor/peso), isto é, o maior valor por quilo.**

- **Por que funciona? Cada fração de capacidade gasta compra o maior retorno financeiro possível. Quando o espaço restante não comporta o próximo item inteiro, pega-se apenas a fração exata para lotar a mochila sem deixar desperdício. Não é um matroide, é um problema contínuo/linear.**

ALGORITMO 3: CODIFICAÇÃO DE HUFFMAN

```
ALGORITMO Huffman(alfabeto, frequências)
ENTRADA: Símbolos do alfabeto e suas respectivas frequências
SAÍDA:   Raiz da árvore ótima de prefixos

1.  Insira cada caractere em uma FilaDePrioridadeMinima pela sua frequência

2.  ENQUANTO tamanho(FilaDePrioridadeMinima) > 1 FAÇA:
3.      nó_esquerdo ← ExtrairMínimo(FilaDePrioridadeMinima)
4.      nó_direito  ← ExtrairMínimo(FilaDePrioridadeMinima)
5.
6.      Crie novo_nó onde:
7.          novo_nó.frequência ← nó_esquerdo.frequência + nó_direito.frequência
8.          novo_nó.filho_esquerdo ← nó_esquerdo
9.          novo_nó.filho_direito  ← nó_direito
10.
11.     Insira novo_nó na FilaDePrioridadeMinima

12. RETORNE ExtrairMínimo(FilaDePrioridadeMinima)
```

- **Contexto: Compressão de dados sem perda (lossless compression), onde caracteres de um texto aparecem com frequências diferentes (alguns muito comuns, outros raros).**
- **Objetivo: Criar um código binário de tamanho variável livre de prefixos que resulte no menor tamanho total em bits para a mensagem.**

ALGORITMO 3: CODIFICAÇÃO DE HUFFMAN

```
ALGORITMO Huffman(alfabeto, frequências)
ENTRADA: Símbolos do alfabeto e suas respectivas frequências
SAÍDA:   Raiz da árvore ótima de prefixos

1.  Insira cada caractere em uma FilaDePrioridadeMinima pela sua frequência

2.  ENQUANTO tamanho(FilaDePrioridadeMinima) > 1 FAÇA:
3.      nó_esquerdo ← ExtrairMínimo(FilaDePrioridadeMinima)
4.      nó_direito  ← ExtrairMínimo(FilaDePrioridadeMinima)
5.
6.      Crie novo_nó onde:
7.          novo_nó.frequência ← nó_esquerdo.frequência + nó_direito.frequência
8.          novo_nó.filho_esquerdo ← nó_esquerdo
9.          novo_nó.filho_direito  ← nó_direito
10.
11.     Insira novo_nó na FilaDePrioridadeMinima

12. RETORNE ExtrairMínimo(FilaDePrioridadeMinima)
```

Escolha Gulosa:

- **Critério: Retirar repetidamente de uma fila de prioridade mínima (Min-Heap) e fundir os dois nós que possuem as menores frequências de ocorrência.**

- **Por que funciona? Como os nós combinados descem na árvore a cada nova fusão, os caracteres mais raros acabam nas folhas mais profundas (ganham sequências longas de bits), enquanto os caracteres mais frequentes ficam próximos da raiz (ganham códigos curtíssimos de 1 ou 2 bits). Não é um matroide, opera sobre estruturas em árvore / floresta hierárquica.**

ALGORITMO 4: ALGORITMO DE KRUSKAL

```
ALGORITMO Kruskal(vértices, arestas)
ENTRADA: Grafo com vértices e arestas ponderadas
SAÍDA:   Conjunto de arestas da Árvore Geradora Mínima

1.  Ordene todas as arestas em ordem crescente de peso
2.  mst ← conjunto vazio
3.  custo_total ← 0

4.  Inicialize cada vértice em um grupo disjunto próprio (Union-Find)

5.  PARA CADA aresta (u, v) DA lista_ordenada FAÇA:
6.      SE BuscarGrupo(u) ≠ BuscarGrupo(v) ENTÃO:
7.          UnirGrupos(u, v)
8.          Adicione aresta (u, v) ao conjunto mst
9.          custo_total ← custo_total + aresta.peso

10. RETORNE mst, custo_total
```

- **Contexto: Projetos de infraestrutura de redes (telecomunicações, estradas, encanamento) modelados por um grafo conexo e ponderado.**
- **Objetivo: Conectar todos os vértices do grafo utilizando o menor número de conexões (V−1 arestas) com o menor custo total possível, sem formar ciclos.**

ALGORITMO 4: ALGORITMO DE KRUSKAL

```
ALGORITMO Kruskal(vértices, arestas)
ENTRADA: Grafo com vértices e arestas ponderadas
SAÍDA:   Conjunto de arestas da Árvore Geradora Mínima

1.  Ordene todas as arestas em ordem crescente de peso
2.  mst ← conjunto vazio
3.  custo_total ← 0

4.  Inicialize cada vértice em um grupo disjunto próprio (Union-Find)

5.  PARA CADA aresta (u, v) DA lista_ordenada FAÇA:
6.      SE BuscarGrupo(u) ≠ BuscarGrupo(v) ENTÃO:
7.          UnirGrupos(u, v)
8.          Adicione aresta (u, v) ao conjunto mst
9.          custo_total ← custo_total + aresta.peso

10. RETORNE mst, custo_total
```

Escolha Gulosa (Focada em Arestas):

- **Critério: Examinar as arestas em ordem global crescente de peso e selecionar a aresta mais barata que conecta componentes desconexas, descartando arestas que fechem ciclo.**

- **Por que funciona? Baseia-se no matroide gráfico. Constrói a solução aceitando progressivamente as conexões mais econômicas do mapa inteiro; com o auxílio da estrutura Union-Find, evita-se qualquer redundância cíclica. É um matroide perfeito.**

ALGORITMO 5: ALGORITMO DE PRIM

```
ALGORITMO Prim(grafo, vértice_origem)
ENTRADA: Lista de adjacência do grafo e vértice de início
SAÍDA:   Custo total da Árvore Geradora Mínima

1.  Marque todos os vértices como NÃO visitados
2.  Inicialize FilaDePrioridadeMinima vazia
3.  custo_total ← 0

4.  Insira na Fila a tupla (peso = 0, vértice = vértice_origem, pai = nulo)

5.  ENQUANTO FilaDePrioridadeMinima NÃO estiver vazia FAÇA:
6.      (peso, u, pai) ← ExtrairMínimo(FilaDePrioridadeMinima)
7.
8.      SE u já foi visitado ENTÃO:
9.          Avance para a próxima iteração
10.
11.     Marque u como visitado
12.     custo_total ← custo_total + peso
13.
14.     PARA CADA vizinho v de u NÃO visitado FAÇA:
15.         Insira na Fila a tupla (peso(u, v), vértice = v, pai = u)

16. RETORNE custo_total
```

- **Contexto: O mesmo problema de infraestrutura de redes do Kruskal (encontrar a Árvore Geradora Mínima de um grafo ponderado).**
- **Objetivo: Interligar todos os vértices com o custo total mínimo, mas expandindo a rede a partir de um ponto inicial fixo (uma central ou cidade-sede).**

ALGORITMO 5: ALGORITMO DE PRIM

```
ALGORITMO Prim(grafo, vértice_origem)
ENTRADA: Lista de adjacência do grafo e vértice de início
SAÍDA:   Custo total da Árvore Geradora Mínima

1.  Marque todos os vértices como NÃO visitados
2.  Inicialize FilaDePrioridadeMinima vazia
3.  custo_total ← 0

4.  Insira na Fila a tupla (peso = 0, vértice = vértice_origem, pai = nulo)

5.  ENQUANTO FilaDePrioridadeMinima NÃO estiver vazia FAÇA:
6.      (peso, u, pai) ← ExtrairMínimo(FilaDePrioridadeMinima)
7.
8.      SE u já foi visitado ENTÃO:
9.          Avance para a próxima iteração
10.
11.     Marque u como visitado
12.     custo_total ← custo_total + peso
13.
14.     PARA CADA vizinho v de u NÃO visitado FAÇA:
15.         Insira na Fila a tupla (peso(u, v), vértice = v, pai = u)

16. RETORNE custo_total
```

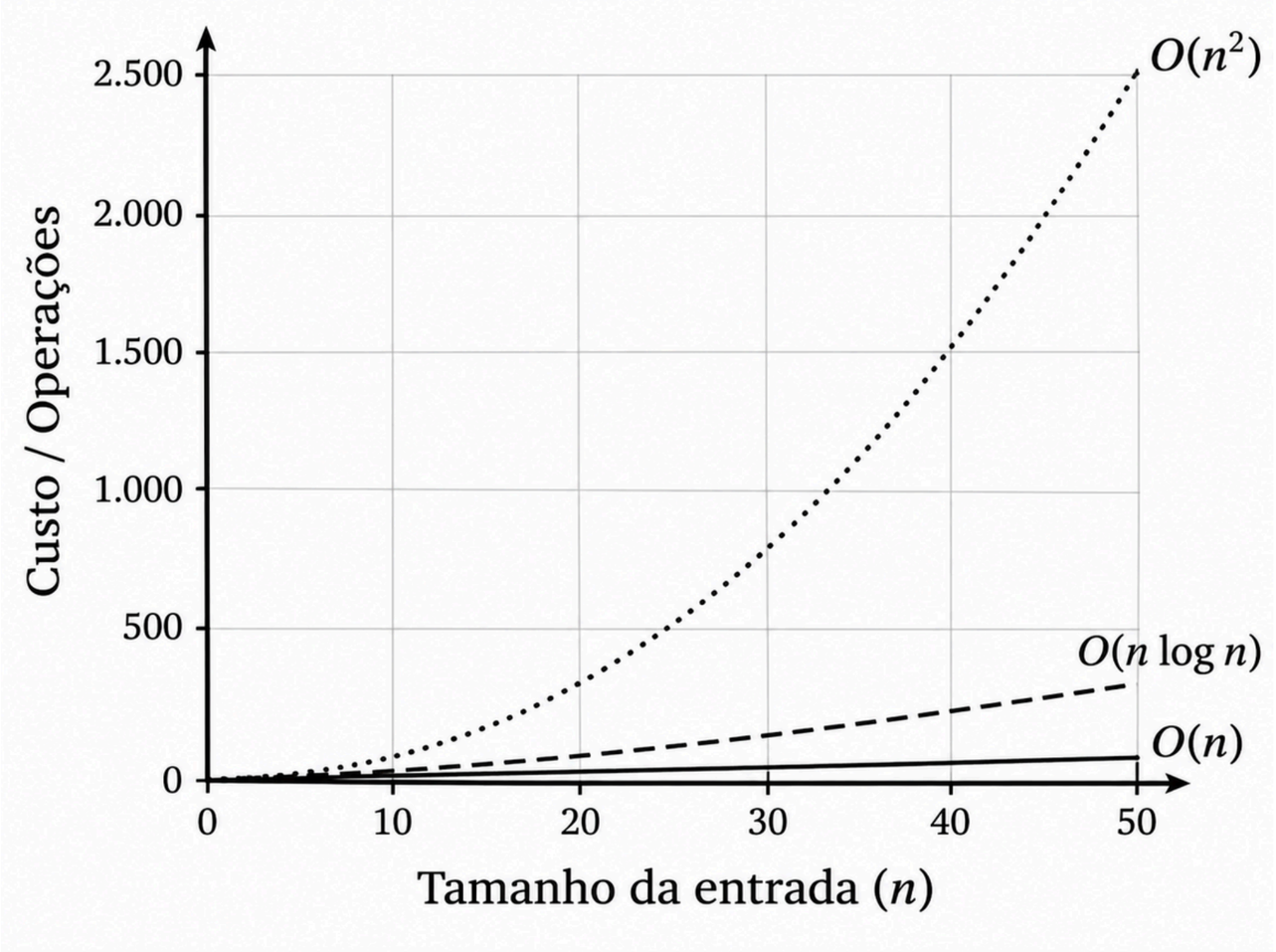
Escolha Gulosa (Focada em Vértices / Corte):

- **Critério: A partir da árvore já existente, selecionar a aresta mais leve que cruza a fronteira (corte) ligando um vértice já visitado a um vértice vizinho ainda não visitado.**

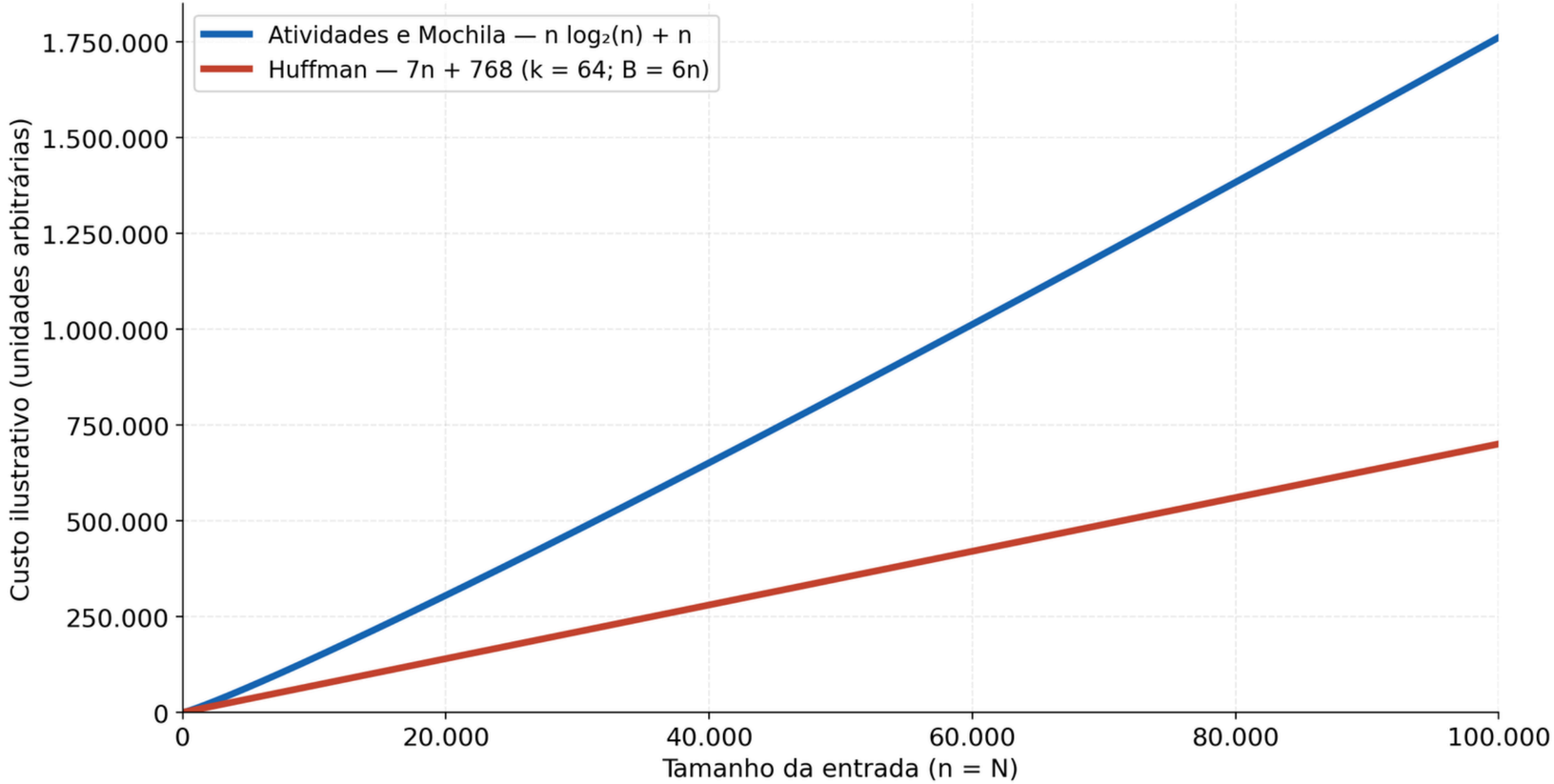
- **Por que funciona? Teorema do Corte: para qualquer divisão dos vértices em dois conjuntos, a aresta mais barata que conecta os dois lados obrigatoriamente pertence a alguma Árvore Geradora Mínima. A árvore cresce como uma mancha contínua, garantindo conexão e evitando ciclos pelo simples rastreio de vértices visitados. Resolve o mesmo problema do Matroide Gráfico, mas não percorre o matroide de forma puramente independente.**

ANÁLISE ASSINTÓTICA: TEMPO E ESPAÇO

- Tempo: quantidade de operações conforme a entrada cresce.
- Espaço: quantidade de memória utilizada.
- O: limite superior.
- Ω: limite inferior.
- Θ: limite justo.



COMPLEXIDADE: ATIVIDADES, MOCHILA E HUFFMAN



n: número de atividades ou itens
k: quantidade de símbolos distintos
B: quantidade de bits gerados
h: altura da árvore de Huffman

Algoritmo	Tempo	Principal custo
Seleção de Atividades	O(n log n)	Ordenação
Mochila Fracionária	O(n log n)	Ordenação
Huffman	O(N + k log k + k² + B)	Heap e geração dos códigos

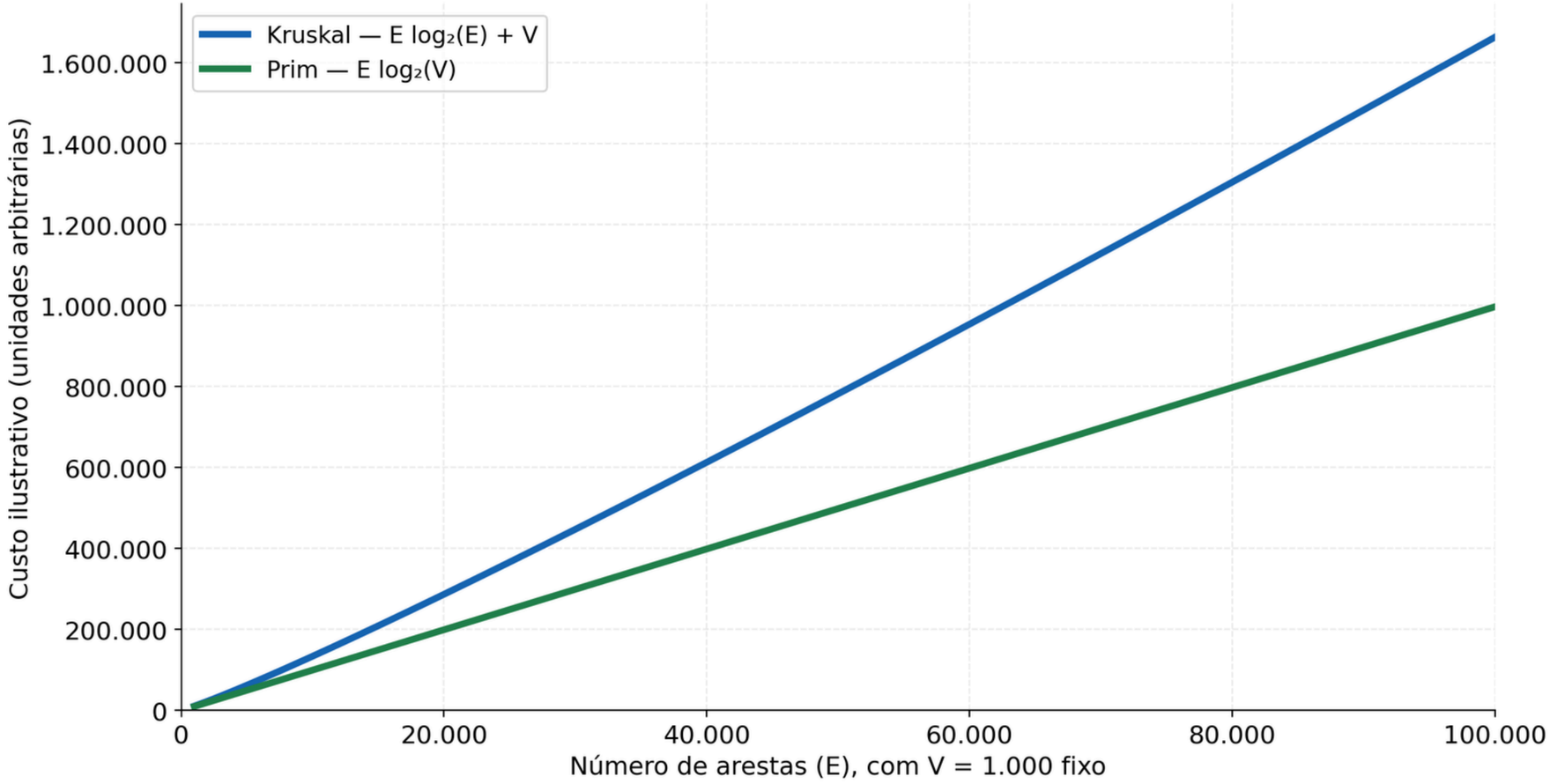
KRUSKAL × PRIM: COMPARAÇÃO DE CUSTOS

01

O(E log E + V)
Ordena todas as arestas.
Utiliza Union-Find para evitar ciclos.
Espaço: Θ(V + E)

02

O(E log V)
Expande uma árvore a partir de um vértice.
Utiliza fila de prioridade mínima.
Espaço: Θ(V + E)



V: número de vértices
E: número de arestas

COMPARAÇÃO FINAL DAS COMPLEXIDADES

Algoritmo	Tempo	Espaço
Atividades	$O(n \log n)$	$\Theta(n)$
Mochila	$O(n \log n)$	$\Theta(n)$
Huffman	$O(N + k \log k + k^2 + B)$	$O(k + L + h^2 + B)$
Kruskal	$O(E \log E + V)$	$\Theta(V + E)$
Prim	$O(E \log V)$	$\Theta(V + E)$

n: número de atividades ou itens
N: tamanho do texto original
k: quantidade de símbolos distintos
B: quantidade de bits gerados
L: soma dos comprimentos dos códigos
h: altura da árvore de Huffman
V: número de vértices
E: número de arestas

QUANDO A ESCOLHA LOCAL GARANTE O ÓTIMO GLOBAL?

RIGOR MATEMÁTICO:

- Garantia de Otimalidade: Exige Propriedade da Escolha Gulosa + Subestrutura Ótima.
- teorema de Rado–Edmonds: Define nos Matroides a fronteira exata onde o ótimo local converge sem backtracking.



BIFURCAÇÃO DO MOTOR GULOSO:

- **Execução Iterativa:** Ramo único de busca com memória auxiliar mínima Θ(1).
- **Motor Estático (Vetor Ordenado):** Custo O(n log n) e alta localidade de cache (Atividades, Mochila, Kruskal)**.**
- **Motor Dinâmico (Min-Heap):** Extração em O(log n) para custos variáveis (Huffman e Prim).

VALIDAÇÃO EXPERIMENTAL (C++20) E FRONTEIRAS DE FALHA

CONVERGÊNCIA NOS ESTUDOS DE CASO:

- **Kruskal vs. Prim:** Caminhos opostos (Union-Find vs. Corte em Heap) convergindo para a mesma MST (Custo = 14).
- **Huffman:** Compressão real de 73,86% nos bits sem perda de dados.



ONDE O PARADIGMA FALHA? (A BARREIRA DISCRETA):

- **Mochila Fracionária (Ótimo: R$ 240) vs. Mochila 0-1 (Falha: R$ 160 contra R$ 220).**
- **Causa:** Restrições discretas violam a propriedade de troca e geram capacidade ociosa, exigindo Programação Dinâmica.

GARGALOS DE ARQUITETURA E VISÃO FINAL

ALÉM DO BIG-O (IMPACTO NO HARDWARE):

- **Localidade de Cache:** Vetores contíguos (Kruskal/Atividades) superam estruturas dispersas na prática.
- **Alocação Dinâmica (Huffman):** Nós dispersos na memória geram cache misses na árvore.
- **Inserção Preguiçosa (Prim):** Acelera grafos esparsos, mas eleva a fila para O(E) em grafos densos.

VEREDITO FINAL

O paradigma guloso transfere o esforço da CPU para a prova matemática prévia, entregando algoritmos rápidos, enxutos e fundamentais para aproximações NP-Difíceis.

OBRIGADO PELA ATENÇÃO!!!!

REFERÊNCIAS

[1] T. H. CORMEN, C. E. LEISERSON, R. L. RIVEST, AND C. STEIN, ALGORITMOS: TEORIA E PRÁTICA, 4ª ED. RIO DE JANEIRO: GEN LTC, 2022. (CAPS. 16 E 21).

[2] D. A. HUFFMAN, "A METHOD FOR THE CONSTRUCTION OF MINIMUM-REDUNDANCY CODES," PROCEEDINGS OF THE IRE, VOL. 40, NO. 9, PP. 1098–1101, SEPT. 1952.

[3] J. B. KRUSKAL, "ON THE SHORTEST SPANNING SUBTREE OF A GRAPH AND THE TRAVELING SALESMAN PROBLEM," PROCEEDINGS OF THE AMERICAN MATHEMATICAL SOCIETY, VOL. 7, NO. 1, PP. 48–50, FEB. 1956.

[4] R. C. PRIM, "SHORTEST CONNECTION NETWORKS AND SOME GENERALIZATIONS," THE BELL SYSTEM TECHNICAL JOURNAL, VOL. 36, NO. 6, PP. 1389–1401, NOV. 1957.

[5] J. EDMONDS, "MATROIDS AND THE GREEDY ALGORITHM," MATHEMATICAL PROGRAMMING, VOL. 1, NO. 1, PP. 127–136, DEC. 1971.

[6] T. H. CORMEN, C. E. LEISERSON, R. L. RIVEST, AND C. STEIN, ALGORITMOS: TEORIA E PRÁTICA, 3ª ED. RIO DE JANEIRO: ELSEVIER, 2012. (CAPS. 15 E 16)

[7] H. WHITNEY, "ON THE ABSTRACT PROPERTIES OF LINEAR DEPENDENCE," AMERICAN JOURNAL OF MATHEMATICS, VOL. 57, NO. 3, PP. 509–533, 1935.

[8] R. RADO, "A NOTE ON INDEPENDENCE FUNCTIONS," PROCEEDINGS OF THE LONDON MATHEMATICAL SOCIETY, VOL. S3-7, NO. 1, PP. 300–320, 1957.

[9] J. OXLEY, MATROID THEORY, 2ND ED. OXFORD: OXFORD UNIVERSITY PRESS, 2011.

[10] P. F. S. SILVA, F. C. NASCIMENTO, F. SILVA, D. E. VIEIRA, A. R. GONÇALVES, AND P. H. S. COSTA, "TRABALHO AEDS II – ALGORITMOS GULOSOS EM C++ MODERNO (C++20 MODULES)," REPOSITÓRIO GITHUB, 2026. DISPONÍVEL EM: HTTPS://GITHUB.COM/PEDROFRANCISCO30/TRABALHO-AEDS-II-ALGORITMOS-GULOSOS

DUVIDAS?

